

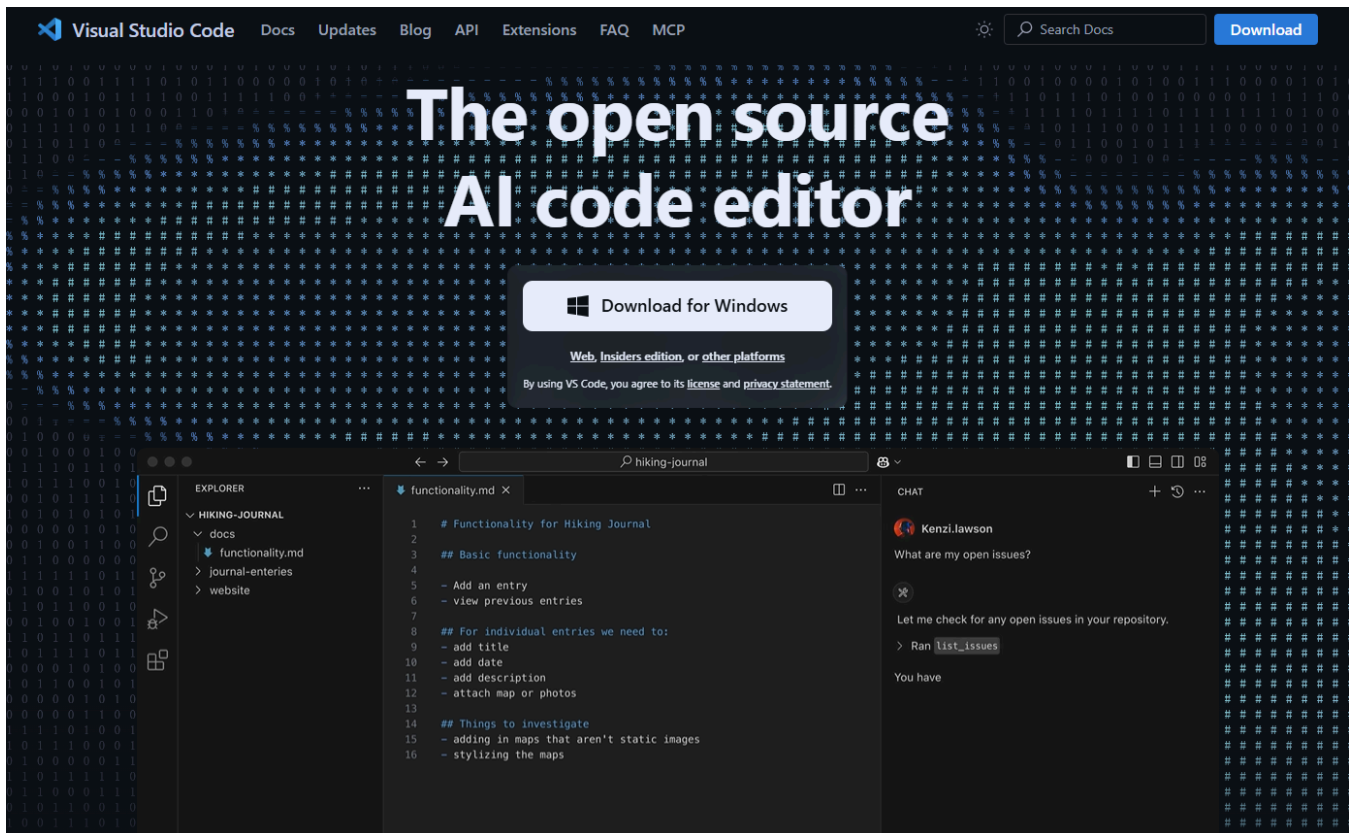
3. ROS2 IDE Setup

Note: The accompanying reference video uses the ROS Humble version, for reference only

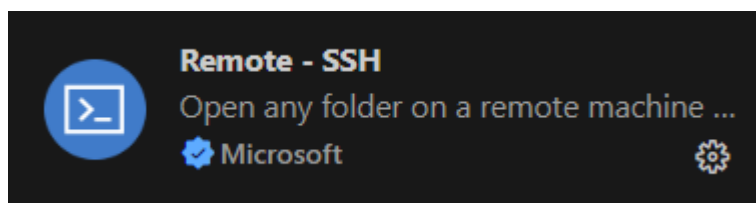
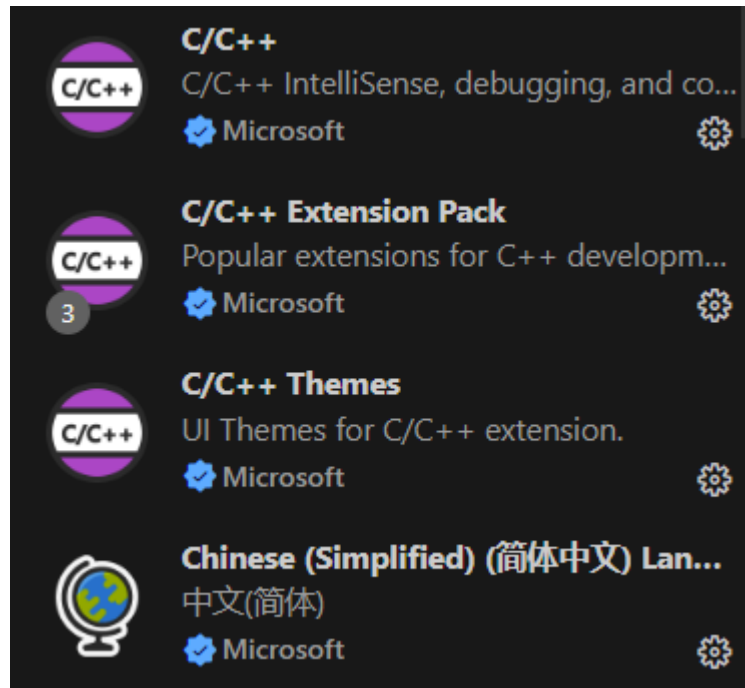
In theory, you can write basic ROS2 programs with just a text editor in ROS2. However, to improve development efficiency, you can install the integrated development environment VSCode.

1. Using VSCode

- Search for VSCode in your browser, download the installer for your platform, and install it.

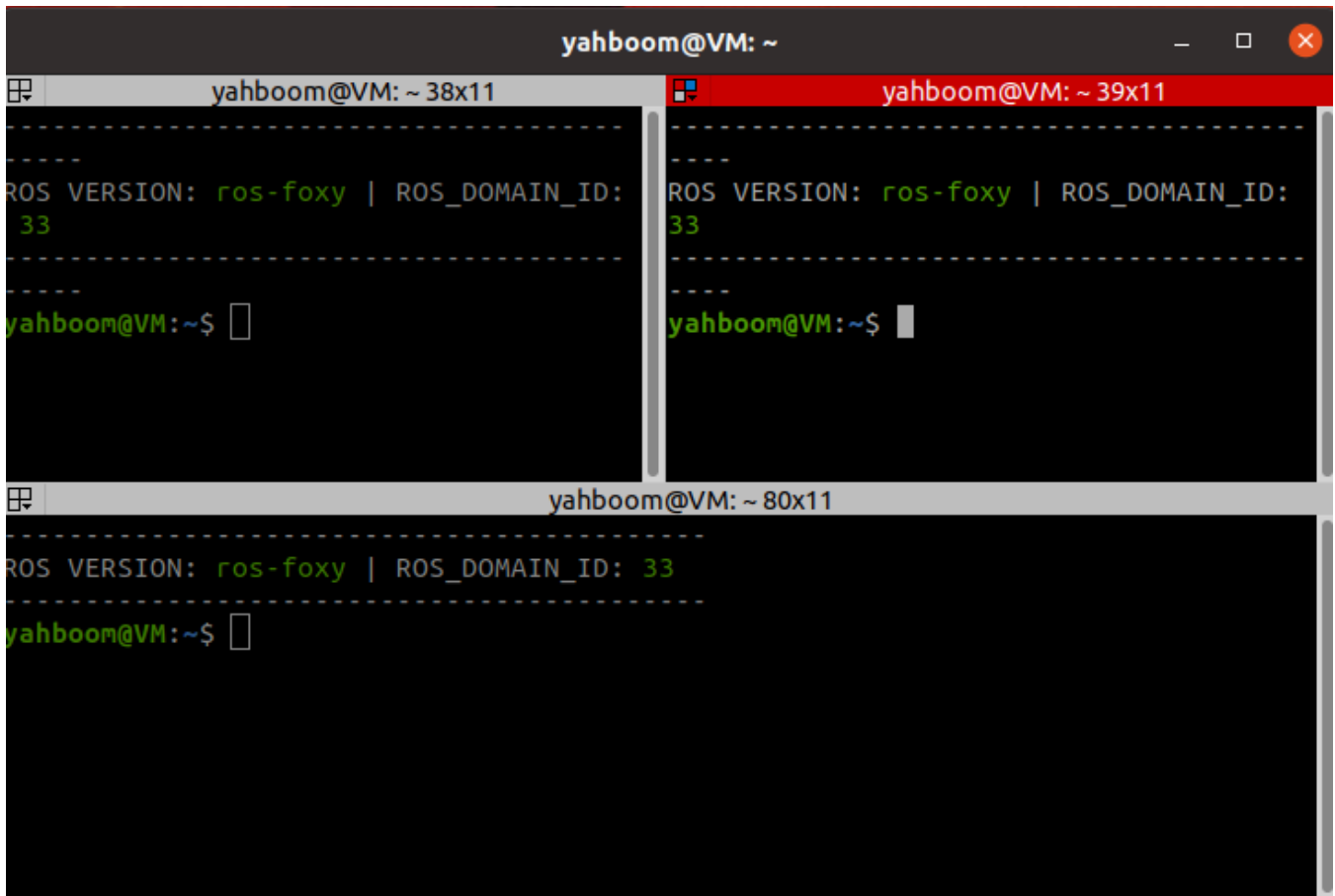


- Install commonly used extensions to improve work efficiency.



2. Using Terminal

In ROS2, you need to use the terminal frequently. We recommend a terminal called Terminator. The effect is as follows:



2.1. Installation

```
sudo apt install terminator
```

2.2. Launch

Press **Ctrl+Alt+T** to launch

2.3. Terminator Common Shortcuts

Operations within the same tab:

Alt+Up	// Move to the terminal above
Alt+Down	// Move to the terminal below
Alt+Left	// Move to the terminal on the left
Alt+Right	// Move to the terminal on the right
Ctrl+Shift+O	// Split terminal horizontally
Ctrl+Shift+E	// Split terminal vertically
Ctrl+Shift+Right	// Move split bar right in vertically split terminal
Ctrl+Shift+Left	// Move split bar left in vertically split terminal
Ctrl+Shift+Up	// Move split bar up in horizontally split terminal
Ctrl+Shift+Down	// Move split bar down in horizontally split terminal
Ctrl+Shift+S	// Hide/show scrollbar
Ctrl+Shift+F	// Search
Ctrl+Shift+C	// Copy selected content to clipboard
Ctrl+Shift+V	// Paste clipboard content here

```
Ctrl+Shift+W          // Close current terminal
Ctrl+Shift+Q          // Exit current window, all terminals in this window will be
closed
Ctrl+Shift+X          // Maximize current terminal
Ctrl+Shift+Z          // Maximize current terminal and enlarge font
Ctrl+Shift+N or Ctrl+Tab // Move to next terminal
Ctrl+Shift+P or Ctrl+Shift+Tab // Move to previous terminal
```

Operations between tabs:

```
F11                  // Toggle fullscreen
Ctrl+Shift+T         // Open a new tab
Ctrl+PageDown        // Move to next tab
Ctrl+PageUp          // Move to previous tab
Ctrl+Shift+PageDown  // Swap current tab with the next tab
Ctrl+Shift+PageUp    // Swap current tab with the previous tab
Ctrl+Plus (+)        // Increase font size
Ctrl+Minus (-)       // Decrease font size
Ctrl+Zero (0)        // Reset font to original size
Ctrl+Shift+R         // Reset terminal state
Ctrl+Shift+G         // Reset terminal state and clear screen
Super+g              // Bind all terminals so input goes to all terminals
Super+Shift+G        // Unbind
Super+t              // Bind all terminals in current tab, input to one terminal
auto-copies to others
Super+Shift+T        // Unbind current tab
Ctrl+Shift+I         // Open a new window using the same process as the original
window
Super+i              // Open a new window using a different process from the
original window
```

3. Using Git

3.1. Installation

In daily work, since it's team collaboration involving version control, Git is an essential skill. Git is a free and open-source distributed version control system. Install Git on Ubuntu:

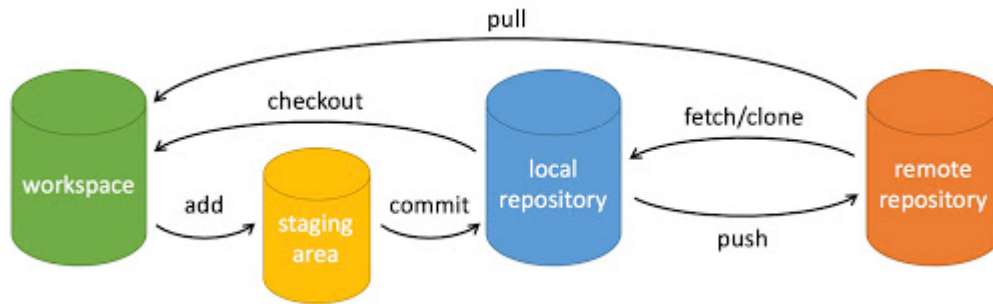
```
sudo apt install git
```

3.2. Git Basic Operations

Git's job is to create and save snapshots of your project and compare them with subsequent snapshots.

This chapter introduces commands for creating and committing project snapshots.

The six most commonly used Git commands are: **git clone**, **git push**, **git add**, **git commit**, **git checkout**, and **git pull**. We will cover them in detail later.



Explanation:

- workspace: Working directory
- staging area: Staging area / cache area
- local repository: Version repository or local repository
- remote repository: Remote repository

A simple operation sequence:

```
$ git init
$ git add .
$ git commit
```

- git init - Initialize a repository.
- git add . - Add files to the staging area.
- git commit - Add staged content to the repository.

3.2.1. Create Repository Commands

The table below lists Git commands for creating repositories:

Command	Description
<code>git init</code>	Initialize a repository
<code>git clone</code>	Copy a remote repository, i.e., download a project.

3.2.2. Commit and Modify

Git's job is to create and save snapshots of your project and compare them with subsequent snapshots.

The table below lists commands for creating and committing project snapshots:

Command	Description
<code>git add</code>	Add files to staging area
<code>git status</code>	View current repository status, showing changed files.
<code>git diff</code>	Compare file differences between staging area and working directory.

Command	Description
<code>git commit</code>	Commit staging area to local repository.
<code>git reset</code>	Revert to a previous version.
<code>git rm</code>	Delete files from staging area and working directory.
<code>git mv</code>	Move or rename files in working directory.
<code>git checkout</code>	Switch branches.
<code>git switch</code> (Introduced in Git 2.23)	Switch branches more clearly.
<code>git restore</code> (Introduced in Git 2.23)	Restore or undo file changes.

3.2.3. Commit Log

Command	Description
<code>git log</code>	View historical commit records
<code>git blame <file></code>	View historical modification records of a file in list format

3.2.4. Remote Operations

Command	Description
<code>git remote</code>	Remote repository operations
<code>git fetch</code>	Fetch code from remote repository
<code>git pull</code>	Download and merge remote code
<code>git push</code>	Upload and merge remote code

For more Git commands, type `git --help` in the terminal to view the help documentation.